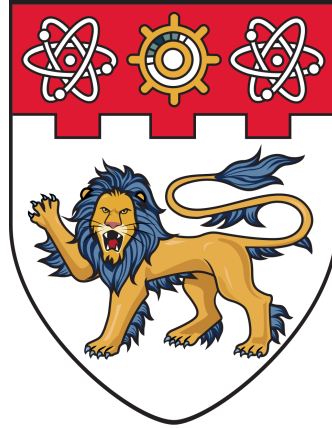


NANYANG TECHNOLOGICAL UNIVERSITY



**IMPROVING ROBUSTNESS OF MALWARE CLUSTERING
USING PROBABILISTIC SUFFIX TREES**

Gao Linyue

College of Computing and Data Science

2026

NANYANG TECHNOLOGICAL UNIVERSITY

CCDS25-0508

**IMPROVING ROBUSTNESS OF MALWARE CLUSTERING USING
PROBABILISTIC SUFFIX TREES**

Submitted in Partial Fulfilment of the Requirements
for the Degree of Bachelor of Engineering in Computer Science
of the Nanyang Technological University

by

Gao Linyue

College of Computing and Data Science

2026

Abstract

Malware threat actors frequently employ obfuscation techniques to manipulate execution traces and evade detection systems. To understand these emerging malware threats, robust behavioural representations are crucial for effective malware clustering. This study evaluates the efficacy of Probabilistic Suffix Trees (PSTs) in generating sequence-based behavioural models for robust malware clustering under synthetic obfuscation and evaluates the framework’s ability to maintain accurate family-level clusters. The findings reveals that while PSTs are able to yield a feasible, probabilistic representation of malware sequences, they are not able to reliably maintain family level taxonomy within clusters. Experimental results observed a non-monotonicity between distance correlation values and noise levels, indicating their inability to reliably withstand structural manipulation. It is concluded that the partial preservation of structure at lower stochastic noise rates suggests viability of using PSTs for behavioural modelling and clustering, though achieving a higher level of clustering robustness would require the incorporation of real obfuscated data for a more representative evaluation, and domain expertise in situations where datasets are sparse and realistic modelling of clean or obfuscated traces are required.

Acknowledgments

I would like to express my deepest gratitude to my supervisor, Dr. Huang Shell Ying, for giving me the opportunity to work on this project. Her constant guidance and support throughout this journey have helped me deepen my understanding of data structures and their applications, while also broadening my exposure to malware research.

Contents

Abstract	i
Acknowledgments	ii
Contents	iii
List of Tables	v
List of Figures	vi
1 Introduction	1
1.1 Background	1
1.2 Objective and Scope	3
1.3 Report Organization	3
2 Literature Review	4
2.1 Conventional Signature-based Techniques	4
2.2 Limitations of Conventional Methods	6
2.3 Deep Learning Approaches	7
2.4 Applications of Suffix Trees	8
3 Implementation Design	10
3.1 Sequence Modelling Framework	10
3.2 Implementation of PSTs	12
4 Experimental Methodology	18
4.1 Experimental Setup	18
4.2 Dataset Patterns and Generation	19

4.3	Testing Procedure	20
4.4	Performance Measurement	21
5	Results and Analysis	22
5.1	Iterative Training and Clustering	22
5.2	Synthetic Obfuscation.....	23
5.3	Threats to Validity.....	24
6	Conclusion	26
6.1	Future Work	26
	Appendix	33

List of Tables

4.2.1 Distribution of Malware Families	19
5.1.1 Mean ARI, NMI, and Silhouette Score	22
5.2.1 Obfuscated ARI, NMI	23
5.2.2 Partition ARI, NMI with Pearson and Spearman Correlations.....	23
6.1.1 ARI, NMI, and Silhouette Scores	36

List of Figures

3.2.1 Suffix-parent structure	13
6.1.1 Family-cluster proportion heatmaps for Runs 1 and 2.....	33
6.1.2 Family-cluster proportion heatmaps for Runs 3, to 5.	34
6.1.3 Family-cluster proportion heatmaps for Runs 6, to 8.	35
6.1.4 Family-cluster proportion heatmaps for Runs 9 and 10.	36

Chapter 1

Introduction

1.1 Background

Motivation

Malware, abbreviated as malicious software, refers to any software program intentionally created to harm computer systems and networks. It encompasses an extensive array of adversarial and intrusive programs, such as viruses, worms, ransomware, and rootkits, all of which are designed to compromise the confidentiality, integrity, and availability of data [1]. Often times, the consequences of successful malware attacks range from financial losses and large-scale data breaches to the disruption of critical services such as transportation and healthcare [2]. With a growing global reliance on technology in this digitalization era [3], there is a prominent need for robust detection and classification of malware families [4].

Over the decade, new malware samples have proliferated to 1.3 billion, with an estimated 9 million appearing monthly [5]. Given the sheer volume of malware, the approach to malware analysis requires scalability. To address this, clustering is utilized to condense these large, unstructured datasets into a few manageable structured groups based on inherent similarities identified between samples. This approach enables a more efficient resource allocation, allowing malware analysts to either apply broad fixes to entire clusters of known variants or prioritize the investigation of smaller, unique clusters that

may indicate emerging threats.

Several clustering methods such as feature-based and graph-based have been used in previous studies [6], [7]. In feature-based clustering using n -grams, a fixed n -gram size is chosen, and each malware sample is represented by the frequency of all n -grams that appear within the sample. This method clusters on the basis of statistical counts, and loses variable length patterns should a malware contain a distinct sequence that is larger than the determined n -gram size. In graph-based clustering using graph edit distances (GED), each malware sequence is represented by a unique graph. The minimum number of node insertion, deletion, or substitution required to transform one call graph into another is then used as a measure of similarity for clustering. However, the computation of GED has a high complexity, and may be impractical when dealing with large datasets.

Other studies to cluster malware have represented malware sequences using generalized suffix tree (GST) structures [8], [9]. Multiple sequences can be represented within a single tree structure, constructed in linear time, while preserving entire sequence lengths without losing context. As a result, shared subsequences are revealed without high complexity pairwise comparisons. While GSTs are effective for exact sequence matching problems [10], their suitability for clustering is limited in obfuscation-prone contexts, since even minor sequence modifications can distort similarity and compromise clustering accuracy.

Many polymorphic or metamorphic malware utilize obfuscation techniques to scramble code and conceal syntactic signatures to evade conventional signature-based detection methods [11]. They do not have consistent code structures despite sharing similar functionality and behaviour. Therefore, a salient point to highlight is that a static code-level analysis will be less effective in the clustering of malware samples. Conversely, behavioural attributes like system calls can be observed during program execution and characterizes malware behaviour at the OS level. Dynamic analysis of such attributes offers greater semantic insight and generally yields a higher accuracy [12], [13].

However, dynamic system call analysis does not guarantee complete resistance, as similar obfuscation strategies, like the insertion of dummy system calls, substitution, and

reordering or interleaving can still undermine the effectiveness of clustering malware families [9].

1.2 Objective and Scope

The objective of this study aims to improve behaviour-based clustering robustness under obfuscation while keeping up with the ever-growing malware corpora.

The scope examines an alternative probabilistic suffix tree (PST) structure proposed by Gabadinho et al [14], and extends it in the context of malware clustering. It incorporates probabilistic sequencing of dynamic features, then clusters sequences based on classification scores generated by PSTs.

PSTs are n -ary tree-based probabilistic models that represent all observed sequences up to length L and assign probabilities to the next state given previous states [15]. This allows for context preservation while also enabling sequence patterns to be modelled in a probabilistic way, potentially addressing the limitation of GSTs in obfuscation-prone contexts and improving clustering robustness under obfuscation.

1.3 Report Organization

Chapter 2 provides a comprehensive literature review on malware analysis, contrasting conventional signature-based techniques with deep learning approaches, while also introducing practical applications using suffix tree structures. Chapter 3 details the proposed PST framework and its implementation. Chapter 4 presents the experimental methodology. Chapter 5 discusses the implications of these findings, and Chapter 6 proposes directions for future work.

Chapter 2

Literature Review

2.1 Conventional Signature-based Techniques

Conventional signature-based techniques for malware detection and analysis span from static analysis, dynamic analysis, and the integration of classical machine learning (ML). They rely on identifying predefined patterns or “signatures” within file executables or dynamic behaviours that indicate malicious activity. While such techniques are highly effective for identifying known malware families, their performance are limited against emerging malware variants that utilize semantic-preserving obfuscation techniques. Their effectiveness also depends heavily on the selection of discriminating features which requires the need for expert knowledge [16].

Static Analysis

Static analysis inspects the binary executables of a malware sample without execution. Within the executables, common discriminating features such as operation codes (opcodes), byte n-grams, and portable executable (PE) headers and metadata are extracted, which are then matched against a database of known malware signatures. A common technique in static analysis examines the opcode sequences in the malware binaries [17], [18]. These sequences are typically transformed into “n-grams”, which are contiguous sub-sequences of n opcodes. N-grams frequencies are then compared against a signature database. Apart from opcodes, n-grams can also be used for raw binary bytes [19],

[20]. Alternative studies that look at PE features extract specific structural features that reveal a file's import and export capabilities, which hints at likely behaviours without execution [21], [22].

However, the accuracy of static techniques can be compromised under packing, polymorphism, and metamorphism. These 3 obfuscation techniques break literal signature matches and shift static feature distributions, potentially introducing false negatives and making it difficult to accurately identify malware samples [23].

Dynamic Analysis

Dynamic analysis involves the process of executing malware samples in a controlled environment, like a virtual machine or a sandbox, to monitor runtime behaviour. It circumvents the issue of static code-level disguises and captures behavioural signatures such as system-calls and API traces. System-call sequences are widely used as it is associated with the interaction between malware and the operating system [9], [15], [24]. Specific system-calls to open files, modify system settings, or send data over network can expose patterns in malicious behaviour, which can be used to generate unique signatures or classify a known malware family. An example of higher-order motifs within a sequence like “open, write, send” define procedural regularities that are comparatively invariant to syntactic obfuscation. Even if these sub-sequences are subjected to tiny re-orderings or paddings of dummy instructions, n-perms allows for the tolerance of such slight obfuscation, maintaining robust detection and classification [25].

Nevertheless, dynamic analysis faces challenges dealing with evasive malware types that are able to suppress their behaviour under observation in virtual environments [26].

Integration of Classical Machine Learning

With the ever-increasing number of malware variants, classical machine learning have been adopted for efficient malware analysis. Supervised learning models such as support vector machines (SVMs), decisions trees, and random forest classifiers are trained over labelled corpora, using either static or dynamic features [27], [28]. They

are then used to classify new malware variants into existing families.

On the other hand, unsupervised clustering methods such as k-means and hierarchical clustering groups unlabelled samples on the same feature spaces using a similarity index. This helps in identifying emerging malware families and zero-day variants without prior signatures [9], [20], [29].

2.2 Limitations of Conventional Methods

Irrespective of machine learning employed to augment signature-centric pipelines for malware detection and analysis, these conventional signature-based techniques exhibit structural limitations in deployment. Due to the complex and dynamic nature of malware variants, the construction of distinct signatures for different malware families require technical and domain expertise.

Additionally, when using supervised classifiers, feature engineering for training sets likewise demands informed choices, for example, determining n-gram lengths, or certain PE header fields to be analysed. Weak or non-discriminative feature sets can drown strong classifiers in noise, resulting in trained models failing to generalize. Conversely, overly rich sets induce high-dimensional feature representations that are hard to regularize, making the pipeline brittle to dataset shift. As new malware variants proliferate, small perturbations in the feature distribution can lower the performance of downstream classification and clustering [16].

Another critical consideration is the class imbalance between malware and benign-ware samples. Classical ML approaches, supervised and unsupervised, typically perform best with a large, diverse corpora. As such, if presented with insufficient and non-diverse training data, it can result in skewed or biased classification, elevating false negatives. This is a notable concern when it comes to nascent or novel malware families that are less known with a few labelled samples.

2.3 Deep Learning Approaches

Addressing Limitations of Manual Feature Engineering

Motivated by these limitations, malware analysis has progressively turned to deep learning (DL). Among various DL models, convolutional neural networks (CNNs) are frequently used to learn hierarchical representations from malware inputs, replacing manually-engineered features. Despite it being an end-to-end model where final classifier scores are generated in the final SoftMax layer, some studies opted to extract the deep features learnt in the penultimate layers and pass it to a separate margin-based classifier such as SVMs for classification instead [30]-[31]. This hybrid architecture seeks to improve classification performance, as CNNs often misclassified in the case of imbalanced data and obfuscation.

In these pipelines, malware binaries derived from PE files are first mapped into either a greyscale or RGB image. These binaries reveal structural information of the malware, and by representing them as images, it preserves the raw structural and spatial patterns of the executable, making them richer in information compared to traditional handcrafted feature vectors. The images are then fed into CNN models for processing before undergoing deep feature extraction. This addresses the challenge of feature engineering by automating feature extraction, eliminating the need for domain expertise, and improving model generalization [30], [32].

Addressing Limitations of Limited Datasets

For nascent malware families, the scarcity of labelled data has prompted the use of semi-supervised DL models. These models leverage on limited labelled data to generate pseudo-labels for unlabelled data over multiple iterations of training episodes. Using both labelled and pseudo-labelled data, it guides the model's decision boundaries into more generalizable regions, significantly improving performance [33].

In contrast, novel malware families present a more extreme scenario where labelled data is minimal or completely absent. Few-shot DL models further extend upon the idea by enabling effective classification with minimal labelled data. Recent few-shot

approaches adopt a multi-prototype modelling paradigm that relaxes the uni-modality assumption of malware families. By allowing several prototypes to represent a family, the model can capture intra-family heterogeneity, improving classification accuracy in 1 to 5-shot regimes [31], [34].

Ultimately, the integration of DL techniques into malware analysis offers a practical means to maintain robustness of malware classification in the real world context where operational success hinges on the rapid adaptation to new variants [35].

2.4 Applications of Suffix Trees

Suffix trees are sequence-native index structures that support exact substring queries and approximate matching [10], [36], [37]. Originally applied in bioinformatics, molecular biology, and computational biology for genomic motif discovery, sequencing, and matching, suffix trees can be extended to malware analysis when dealing with sequential data like system calls and API traces.

Pre-emptive Approach in Intrusion Detection Systems

Probabilistic suffix trees (PSTs) provide variable-order modelling and likelihood-based scoring, which allows it to predict the next likely event in a sequence based on prior occurrences [15], [38]. Likewise, weighted suffix trees (WSTs) provide probabilistic pattern matching although each individual event is independent and not based on previous contexts [24]. Both PSTs and WSTs can be used to model normal system behaviour while flagging out any deviations as anomalous. This is particularly useful against zero-day attacks or unknown malware that exhibit novel behaviour, as early mitigation measures can be deployed in the event of a potential intrusion before significant damage can be done.

Reactive Approach for Post-hoc Analysis

As opposed to PSTs and WSTs, generalized suffix trees (GSTs) are used to efficiently represent multiple sequences in a single tree structure. By co-indexing multiple sequences, GSTs allow for multi-sequence comparison. This is crucial as the longest

common substrings shared across samples can be revealed regardless of offsets, which is essential when trying to detect similarities between various malware samples [8], [9]. This provides a principled basis for malware family clustering and classification tasks.

In addition, GSTs allow for efficient updates on the suffix tree structure to accommodate new malware samples, yet a critical drawback lies in its inability to scale with expansive datasets. This can be addressed using compressed suffix trees (CSTs), whereby redundant paths within the tree structure are compressed, substantially reducing the space complexities associated with storing and querying extensive malware datasets, while retaining sequence matching functionalities [39].

Chapter 3

Implementation Design

The probabilistic suffix tree (PST) is implemented under the context-tree framework, following the pruning and smoothing scheme of Gabadinho et al. [14].

For context pruning, non-frequent contexts that fall below a minimum support threshold to be considered as statistically significant are discarded. Redundant child contexts that do not provide additional information are also removed.

For probability smoothing, a baseline probability is assigned to any null probabilities such that a zero frequency problem will not nullify the estimation of joint probability.

3.1 Sequence Modelling Framework

Markov Chain Factorization

Given a discrete sequence $x = (x_1, x_2, \dots, x_i)$, the probability of the entire sequence $P(x)$ can be expressed using the chain rule:

$$P(x) = P(x_1) P(x_2 | x_1) P(x_3 | x_1, x_2) \cdots P(x_i | x_1, \dots, x_{i-1}). \quad (1)$$

Each state is therefore conditioned on the observed past (*contexts*). However, as the length i of the sequence increases, the number of possible contexts increases exponentially. Directly modelling the full context for each state becomes impractical

due to space constraints and context sparsity, whereby long contexts occur rarely, and the empirical counts required to reliably estimate long-context distributions $P(X_i \mid \text{ctx})$ are too small.

The Markov Hypothesis

To reduce complexity, the Markov hypothesis states that the full context of all past states can be summarized by a window of the most recent L states for a model of L^{th} order:

$$P(x_i \mid x_{1:i-1}) \approx P(x_i \mid x_{i-L:i-1}). \quad (2)$$

Given a first-order case ($L = 1$), each subsequent state will depend only on the immediately preceding state, yielding:

$$P(x) = P(x_1) P(x_2 \mid x_1) P(x_3 \mid x_2) \cdots P(x_i \mid x_{i-1}). \quad (3)$$

However, constraining the model to a single global L risks under-fitting long-range dependencies when L is small. Conversely, using a large L increases sparsity, which can result in over-fitting.

Variable-Length Markov Chains and Contexts

Variable-length Markov chain (VLMC) models generalize fixed-order Markov chains by allowing the effective memory length to vary by context. Let a context observed before a state be $c = (c_1, \dots, c_k)$, and the length of retained past states be L (with $0 \leq L \leq k$). For each context, the predictive distribution can be approximated using only the preceding L states:

$$P(\sigma \mid c_1, \dots, c_k) \approx \hat{P}(\sigma \mid c_{k-L+1}, \dots, c_k) \quad (4)$$

Pruning of Contexts

VLMC models imply that different patterns require different memory lengths in order to be predictive. Therefore, only useful contexts that contribute additional predictive

information are retained. Before pruning redundant contexts, all contexts up to the maximum length are first extracted as candidate contexts. Each candidate context's conditional distribution is compared to that of its longest suffix. Given a context c , if its predictive distribution is approximately the same as that of its suffix $\text{suf}(c)$, as shown below:

$$\hat{P}(\sigma \mid c) \approx \hat{P}(\sigma \mid \text{suf}(c)), \quad \sigma \in A. \quad (5)$$

then the context is dropped, as retaining c would only increase model complexity without meaningfully altering the next-symbol distribution.

3.2 Implementation of PSTs

The implementation details of a probabilistic suffix tree (PST) for modelling malware behaviour sequences follows that of Gabadinho's framework. However, it extends beyond just the classification of sequences. Classification scores from PSTs are used as embeddings for downstream clustering.

Context Extraction and Counting

Let A denote the set of discrete symbols. Given training sequences $\{x^{(m)}\}$, A is first constructed from all symbols observed across $\{x^{(m)}\}$. The context extraction process then begins, as summarized in Algorithm 1.

Algorithm 1 Enumeration of Context–Symbol Pairs

Require: sequence seq of length n , maximum context depth L

Ensure: list $\mathcal{P}$ of (context, symbol) pairs

```

1:  $\mathcal{P} \leftarrow \emptyset$ 
2: for  $i \leftarrow 0$  to  $n - 1$  do
3:    $d_{\max} \leftarrow \min(L, i)$ 
4:   for  $d \leftarrow 0$  to  $d_{\max}$  do
5:     context  $\leftarrow$  the  $d$  symbols immediately preceding position  $i$ 
6:     symbol  $\leftarrow \text{seq}[i]$ 
7:     Append (context, symbol) to  $\mathcal{P}$ 
8:   end for
9: end for
10: return  $\mathcal{P}$ 

```

For each position i in each sequence, Algorithm 1 extracts all suffix contexts of length $d \in \{0, 1, \dots, \min(L, i)\}$, where each context consists of the d symbols preceding x_i , with $\min(L, i)$ ensuring that the context does not extend beyond the start of the sequence. Each extracted pair $(c^{(m)}, x_i)$ increments an empirical count $N(c, a)$, where $N(c, a)$ denotes the number of times symbol a occurs immediately after context c . The total support of context c is

$$N(c) = \sum_{a \in A} N(c, a). \quad (6)$$

Suffix-Parent Structure

The suffix-parent structure can be represented using Figure 3.2.1. Each context node c is linked to its suffix-parent $\text{suf}(c)$, obtained by removing the oldest symbol in c .

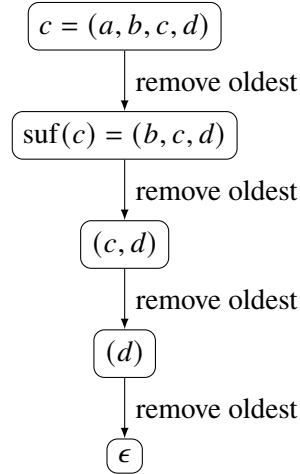


Figure 3.2.1: Suffix-parent structure

Conditional Probability Estimation with Smoothing

For each context c with sufficient support $N(c) \geq n_{\min}$, the empirical conditional distribution is estimated by:

$$\hat{P}(a | c) = \frac{N(c, a)}{N(c)}, \quad a \in A. \quad (7)$$

where $N(c)$ is the total number of occurrences of context c followed by any symbol a . For next-symbol conditional probabilities that do not appear in the training sequences, a null probability is assigned. To avoid an undefined value when computing log-

probabilities, the implementation applies a smoothing scheme with floor parameter $y_{\min} > 0$. Let $k = |A|$, the smoothed distribution will be:

$$\tilde{P}(a \mid c) = (1 - k y_{\min}) \hat{P}(a \mid c) + y_{\min}. \quad (8)$$

Pruning via Relative Change to Suffix-Parent

After estimating smoothed distributions for all candidate contexts, the PST is pruned based on (5) as shown in Algorithm 2.

Algorithm 2 Gain-Based Pruning of Candidate Contexts

Require: probs, dictionary mapping each context c to a probability distribution over symbols in A

Require: parent, dictionary mapping each context c to its suffix parent

Require: gain cut-off τ

Ensure: pruned dictionary probs

```

1:  $\mathcal{K} \leftarrow$  contexts in probs
2:  $\mathcal{C} \leftarrow$  contexts in probs, sorted by decreasing length
3: for each context  $c$  in  $\mathcal{C}$  do
4:    $p \leftarrow \text{parent}[c]$ 
5:   if  $p$  is null or  $p \notin \text{probs}$  then
6:     continue
7:   end if
8:    $\text{childDist} \leftarrow \text{probs}[c]$ 
9:    $\text{parentDist} \leftarrow \text{probs}[p]$ 
10:   $\text{significant} \leftarrow \text{FALSE}$ 
11:  for each symbol  $a$  in  $A$  do
12:    if  $\text{parentDist}[a] = 0$  then
13:       $r \leftarrow \infty$ 
14:    else
15:       $r \leftarrow \text{childDist}[a] / \text{parentDist}[a]$ 
16:    end if
17:    if  $r \geq \tau$  or  $r \leq 1/\tau$  then
18:       $\text{significant} \leftarrow \text{TRUE}$ 
19:      break
20:    end if
21:  end for
22:  if  $\text{significant} = \text{FALSE}$  then
23:    Remove  $c$  from  $\mathcal{K}$ 
24:  end if
25: end for
26: Update probs with  $\mathcal{K}$ 
27: return probs

```

For each context c (processing from child contexts upward), the model compares $\tilde{P}(\cdot | c)$ to $\tilde{P}(\cdot | \text{suf}(c))$. A context is retained only if there exists at least one symbol $a \in A$ such that the ratio

$$\frac{\tilde{P}(a | c)}{\tilde{P}(a | \text{suf}(c))}$$

deviates beyond a cutoff factor $\tau > 1$, i.e.,

$$\frac{\tilde{P}(a | c)}{\tilde{P}(a | \text{suf}(c))} > \tau \quad \text{or} \quad \frac{\tilde{P}(a | c)}{\tilde{P}(a | \text{suf}(c))} < \frac{1}{\tau}.$$

Prediction by Suffix Back-off

Given a context query h , the prediction returns the next-symbol distribution associated with the longest available suffix of h that is retained in the pruned PST. If h does not exist, the model backs off along successive suffixes until a match is found, as shown in Algorithm 3.

Algorithm 3 Prediction Back-Off for Next-Symbol Distribution

Require: context sequence ctx
Ensure: next symbol probability distribution

- 1: **if** probs is empty **or** $|A| = 0$ **then**
- 2: **raise error**
- 3: **end if**
- 4: **while** true **do**
- 5: **if** $\text{ctx} \in \text{probs}$ **then**
- 6: **return** probs[ctx]
- 7: **end if**
- 8: **if** length(ctx) = 0 **then**
- 9: rootDist $\leftarrow$ probs[ϵ]
- 10: **if** rootDist is defined **then**
- 11: **return** rootDist
- 12: **end if**
- 13: $u \leftarrow 1/|A|$
- 14: **return** $\{a \mapsto u \mid a \in A\}$
- 15: **end if**
- 16: ctx $\leftarrow$ suf(ctx)
- 17: **end while**

Scoring via Log-Likelihood

To score a sequence $x_{1:n}$ under a trained PST, Algorithm 4 computes:

$$\log P(x_{1:n}) = \sum_{i=1}^n \log \tilde{P}(x_i \mid c_i), \quad (9)$$

where c_i is the preceding context (up to depth L), and its next-symbol distribution is obtained using the back-off prediction procedure in Algorithm 3.

Algorithm 4 Log-Likelihood of a Sequence

Require: sequence seq of length n

Ensure: log-likelihood value ℓ

```

1:  $\ell \leftarrow 0$ 
2: for  $i \leftarrow 0$  to  $n - 1$  do
3:   context  $\leftarrow \min(L, i)$  symbols immediately preceding position  $i$ 
4:   dist  $\leftarrow \text{PredictionBackOff}(\text{context})$ 
5:    $p \leftarrow \text{dist}[\text{seq}[i]]$ 
6:   if  $p$  is missing or undefined then
7:      $p \leftarrow \varepsilon$ 
8:   end if
9:    $\ell \leftarrow \ell + \log(p)$ 
10: end for
11: return  $\ell$ 

```

Construction of PST Model

PST models are first constructed and pruned using Algorithms 1 and 2.

Algorithm 5 PST Construction

Require: set of API sequences $S = \{s_1, s_2, \dots, s_n\}$, max depth L , gain cutoff τ

Ensure: Pruned context-distribution dictionary $\mathcal{T}$

```

1: Initialize global context counts table  $T \leftarrow \emptyset$ 
2: Initialize suffix-to-parent mapping  $\mathcal{M} \leftarrow \emptyset$ 
3: Initialize context-distribution dictionary  $\mathcal{T} \leftarrow \emptyset$ 
4: for each sequence  $s \in S$  do
5:    $\Sigma \leftarrow \Sigma \cup \{\text{all unique symbols in } s\}$ 
6:   Execute Algorithm 1 to extract all  $(\text{context}, \text{symbol})$  pairs in  $s$  up to depth  $L$ 
7:   Update counts in  $T$  for each pair and suffix-to-parent mapping  $\mathcal{M}$ 
8:   Update probability distribution in  $T$  for each distinct context
9: end for
10:  $\mathcal{T} \leftarrow \text{Execute Algorithm 2}(\mathcal{T}, \tau)$ 
11: return  $\mathcal{T}$ 

```

Clustering using PSTs

Malware sequences are classified using the constructed PST models, with classification scores used as embedding vectors for hierarchical clustering.

Algorithm 6 Behavioural Embedding and Hierarchical Clustering

Require: sequences $s = \{s_1, \dots, s_n\}$, family-specific PST models $m = \{m_1, \dots, m_f\}$

Ensure: Cluster assignments A

```
1: Initialize embedding matrix  $\mathbf{X} \in \mathbb{R}^{n \times f}$ 
2: for  $i \leftarrow 1$  to  $n$  do
3:   for  $j \leftarrow 1$  to  $f$  do
4:      $\ell \leftarrow$  Execute Algorithm 4 to obtain log-likelihood score
5:      $\mathbf{X}_{ij} \leftarrow \ell / \text{length}(s_i)$  for length normalization
6:   end for
7: end for
8:  $\mathbf{X}_{scaled} \leftarrow \text{Standardize}(\mathbf{X})$ 
9:  $\mathcal{Z} \leftarrow \text{WardLinkage}(\mathbf{X}_{scaled})$ 
10:  $A \leftarrow \text{FlatCluster}(\mathcal{Z}, \text{num\_clusters} = k)$ 
11: return  $A$ 
```

Chapter 4

Experimental Methodology

This section details the experimental setup, generation of obfuscated datasets, and the optimization of PST model hyper-parameters such that the behavioural sequential nuances of malware can be captured and translated into discriminative embeddings.

4.1 Experimental Setup

To ensure reproducibility, confounding factors arising from hardware and software variability were explicitly controlled within the experimental environment.

As PST model training involves repeated model fitting during hyper-parameter tuning over a large dataset of variable-length sequences, hardware can play a key difference in runtime performance. Faster hardware allows for more extensive hyper-parameter exploration, and may implicitly lead to improved results. As a result, the study was conducted on the HP Pavilion Laptop 14 with a 12th Gen Intel(R) Core(TM), i7-1255U processor (1.70 GHz), and 16.0 GB of RAM. The hyper-parameter grid size was also fixed across all PST model evaluations. This was to ensure that performance differences were attributable to model behaviour instead of computational advantage.

The software environment used was Python 3.11.9 and Jupyter Notebook 7.4.5. Due to stochastic variation in dataset shuffling and train-validation splitting whenever the local kernel was set up, all sources of randomness were controlled by fixing random seeds.

This was to ensure strict reproducibility of results unless experimental parameters were explicitly modified.

4.2 Dataset Patterns and Generation

Malware API Trace Sequences

PST models were trained using a public malware dataset generated by Cuckoo Sandbox based on Windows OS API calls analysis, serving as a benchmark API-call dataset for Windows PE malware classification [40]. The dataset contains 7107 labelled API trace-call sequences of 8 different malware families, with a relatively equal number of sequences per family, except for *Adware*, as shown in Table 4.2.1.

Table 4.2.1: Distribution of Malware Families

Malware Family	Number of Sequences
Trojan	1001
Downloader	1001
Backdoor	1001
Worms	1001
Virus	1001
Dropper	891
Spyware	832
Adware	379

Obfuscated Datasets

To address the scarcity of publicly available obfuscated datasets, synthetic obfuscation was done on the benchmark dataset. This approach allows for a controlled stress test against common evasion tactics, such as junk-code insertion and control-flow perturbation as obfuscation aggression, represented by noise rates, can be manually controlled. A total of 4 obfuscated datasets were generated with a noise rate of 0.01, 0.05, 0.10, and 0.20. The robustness of PST models can therefore be evaluated against different levels of obfuscation.

Simple obfuscation techniques such as the insertion of encoded dummy API calls and control-flow perturbation via stochastic shuffling of subsequences, were independently

implemented and randomly applied on sequences in the respective datasets, as described in Algorithms 7 and 8 respectively.

Algorithm 7 InsertNoise(sequence, noise_rate)

Require: sequence, noise_rate

Ensure: noisy_sequence

```

1: noisy_sequence  $\leftarrow$  copy of sequence
2: num_noisy_calls  $\leftarrow \lfloor \text{length}(\text{sequence}) \times \text{noise\_rate} \rfloor$ 
3: for  $i \leftarrow 1$  to num_noisy_calls do
4:   insert_position  $\leftarrow$  random integer in  $[0, \text{length}(\text{noisy\_sequence})]$ 
5:   Insert dummy_call into noisy_sequence at insert_position
6: end for
7: return noisy_sequence

```

Algorithm 8 ShuffleSubsequence(sequence, shuffle_rate)

Require: sequence, shuffle_rate

Ensure: noisy_sequence

```

1: noisy_sequence  $\leftarrow$  copy of sequence
2: num_shuffles  $\leftarrow \lfloor \text{length}(\text{sequence}) \times \text{shuffle\_rate} \rfloor$ 
3: for  $i \leftarrow 1$  to num_shuffles do
4:   start_index  $\leftarrow$  random integer in  $[0, \text{length}(\text{noisy\_sequence}) - 2]$ 
5:   end_index  $\leftarrow$  random integer in  $[\text{start\_index} + 1, \text{length}(\text{noisy\_sequence})]$ 
6:   subsequence  $\leftarrow$  noisy_sequence[start_index : end_index]
7:   shuffled_subsequence  $\leftarrow$  random permutation of subsequence
8:   Replace noisy_sequence[start_index : end_index] with shuffled_subsequence
9: end for
10: return noisy_sequence

```

4.3 Testing Procedure

A PST model was trained for each of the malware families on the benchmark dataset using an 80:20 train–test split. This prioritization of the training phase is to ensure that the PST models have sufficient exposure to the diverse API call sequences within each family to provide a statistically significant measure of accuracy. Classification-driven hyper-parameter tuning was conducted across all 8 models to improve discriminative log-likelihood scoring and multi-class classification accuracy. Because multi-class predictions depend on the relative scores across all class-specific generative models, sequential greedy tuning can be order-dependent, whereby the tuning of one family’s

model changes the effective decision boundaries for the remaining families. Exhaustively evaluating all tuning orders is computationally infeasible ($8!$ permutations), hence the tuning order was pre-defined, starting from the family with the lowest classification accuracy.

Following this, each malware sequence in the test set was classified, with likelihood scores across all eight models length-normalized and used as vector embeddings for hierarchical clustering. The number of clusters K was fixed to ensure comparability across runs. The retraining and clustering procedure was repeated for 10 additional runs. The benchmark dataset was shuffled each iteration, while the tuned model hyperparameters were kept fixed. This was done to assess clustering stability by isolating the effect of sampling variation in the training sets. The obfuscated datasets, together with the benchmark dataset, were subsequently clustered using the tuned PST models trained from the first iteration.

4.4 Performance Measurement

To evaluate the extent to which clustering recovers family-level structure, external validity metrics such as Adjusted Rand Index (ARI) and Normalized Mutual Information (NMI) were used. ARI accounts for strict pairwise agreement between predicted clusters and ground-truth labels, while NMI quantifies the reduction in uncertainty regarding a sequence’s true family given its cluster assignment. Global silhouette score was also used to assess average intra-cluster cohesion and inter-cluster separation.

To evaluate robustness under obfuscation, clean-vs-obfuscated ARI and NMI scores were computed. Distance preservation metrics such as Pearson and Spearman correlations were also used to assess structural changes even when cluster assignments differ.

Chapter 5

Results and Analysis

5.1 Iterative Training and Clustering

Clustering results across the 10 independent runs are as summarized in Table 5.1.1.

Table 5.1.1: Mean ARI, NMI, and Silhouette Score			
Metric	Mean $\pm$ Std	Min	Max
ARI	0.0186 $\pm$ 0.0054	0.0127	0.0301
NMI	0.0690 $\pm$ 0.0097	0.0548	0.0845
Global Silhouette	0.2537 $\pm$ 0.0256	0.2036	0.2850

Mean ARI score and NMI score were 0.0186 and 0.0690 respectively, indicating a low external agreement between the clustered partitions and malware family labels. A mean global silhouette score of 0.2537 also suggests limited cluster separation and potential cluster overlaps. This is further substantiated with family–cluster proportion heatmaps (see Appendix 6.1), in which substantial fragmentation of malware families can be observed across clusters, with clear family mixing within clusters. A broad pattern, illustrated by two or three bright vertical bands across rows, appears consistent across all runs. These “macro” clusters contain considerable portions of multiple families, while the remaining mass from each family is distributed across several other clusters. The consistency in observations suggests relative stability in clustering structure under sampling variation despite weak agreement with ground-truth family labels.

When evaluated alongside the small deviations in ARI, NMI, and global silhouette

scores across runs (see Appendix 6.1), it reinforces the idea that family labels are not a good proxy for behavioural similarity. This interpretation is plausible as malware families, though distinct, may exhibit convergent API-call subsequences associated with common operational objectives, such as system reconnaissance, network communication, and file download activity.

5.2 Synthetic Obfuscation

As obfuscation noise is introduced, ARI and NMI values generally deteriorate relative to the clean baseline, as summarized in Table 5.2.1.

Table 5.2.1: Obfuscated ARI, NMI

Noise rate	ARI	NMI
0	0.02629	0.08840
0.01	0.01627	0.05442
0.05	0.00482	0.03640
0.10	0.00745	0.03678
0.20	0.00780	0.03396

At a noise rate of 0.01, ARI decreased from 0.0263 to 0.0163, and further down to 0.0048 at a noise rate of 0.05. NMI decreased from 0.0884 to 0.0544, and to 0.0364 at the respective noise rates. This downward trend suggests that synthetic perturbations further weaken the correspondence between ground-truth family labels and behavioural similarity.

When comparing between clean and obfuscated partitions, their partition ARI and NMI values are as summarized in Table 5.2.2.

Table 5.2.2: Partition ARI, NMI with Pearson and Spearman Correlations

Noise rate	Partition-ARI	Partition-NMI	Pearson	Spearman
0.01	0.21251	0.27968	0.47598	0.44633
0.05	0.12708	0.20860	0.27877	0.22429
0.10	0.10844	0.19870	0.35915	0.26712
0.20	0.13699	0.20162	0.65015	0.57011

Under mild obfuscation of noise 0.01, ARI and NMI partition values are 0.21251 and 0.27968 respectively. These values decreased under more aggressive obfuscation, with ARI partition values falling to 0.127098, then to 0.10844. NMI partition values fell to

0.20860, then to 0.19870 at the respective noise rates. The moderately low ARI and NMI partition values suggests partial but limited stability of cluster membership under obfuscation.

However, to better understand changes in embedding geometry, and to obtain a reliable measure of structural distortion, distance preservation metrics such as Pearson and Spearman correlations are used. They quantify the consistency of pairwise distances and their relative ordering, providing complementary insight into embedding geometry, regardless of whether a small shift in embedding space led to points near cluster boundaries to be reassigned.

As shown in Table 5.2.2 above, Pearson and Spearman correlations between pairwise embedding distances for benchmark and obfuscated sets remain moderately positive across all noise rates. Pearson correlations range between 0.279-0.650, while Spearman correlations range between 0.224-0.570. This implies that the obfuscation process did not completely distort the relative geometry of embeddings.

However, a crucial observation reveals that these correlations do not exhibit a strict monotonic decline following an increase in noise rate. In particular, the obfuscated dataset with a noise rate of 0.20 yielded a significantly higher Pearson correlation of 0.650 and a Spearman correlation of 0.571, as compared to previous noise levels. This behaviour is consistent with the use of random perturbations and suggests that the geometric distortion need not increase monotonically with nominal noise rate.

5.3 Threats to Validity

A key threat to validity arises from the use of stochastic synthetic perturbations for obfuscation. They are not fully representative of real packed or obfuscated malware traces in which obfuscation is done in a 'systematic' structure. Such systematic obfuscation usually preserves local semantic structures where functional calls often remain in close proximity to each other. In contrast, the synthetic method used in this research introduces completely random noise. As a result, these completely random perturbations within the sequences may introduce unpredictable variance in classification

scores by PST models, potentially degrading the quality of clustering embeddings used for downstream clustering. While the modelling of obfuscation in real case scenarios require domain expertise and execution of malware sequences in a controlled sandbox environment, stochastic perturbations can be used as a worst-case scenario whereby PST model robustness is evaluated against a higher entropy obfuscation.

Another limitation is the class imbalance among malware families, with 'Adware' having a significantly lower number of sequences. This is representative of the threat landscape, where 'Adware' is less commonly seen as compared to the other malware families. While the imbalance can be avoided by undersampling larger families, it is undesirable as it drastically reduces the amount of training data available for the other PST models. Likewise, removing 'Adware' reduces the scope of the threat landscape. Therefore, the original distribution is retained to provide a realistic evaluation of the models in the current threat landscape, even if it subsequently affects the training of PST models and the quality of clustering embeddings.

Chapter 6

Conclusion

Overall, this study was able to yield a probabilistic, sequence-based representation of dynamic malware behavioural traces using PSTs. The hyper-parameter tuning of PST models was able to yield a moderate intermediate multi-class classification accuracy of 62%. However, despite optimizing the models' discriminative performance, the log-likelihood scores used as clustering embeddings revealed that the induced clusters were not strongly family-pure, which shows that family labels are not a good proxy for behavioural similarities.

When the PST models were run through the same classification and clustering pipeline on the synthetically obfuscated datasets obtained by independently introducing stochastic noise, the clustering results support a nuanced conclusion aligned with the study objective. The findings observed a non-monotonic relationship between noise rates and distance correlation, which made it insufficient to claim robustness of using PSTs for clustering under obfuscation. Despite this, the partial preservation of structure at lower stochastic noise rates suggests viability of using PSTs for behavioural modelling and clustering.

6.1 Future Work

While PSTs remain a feasible behavioural representation, future research aiming at robust clustering using PSTs under obfuscation likely demands the incorporation of

real obfuscated data for a more representative evaluation. Domain expertise could also prove beneficial in situations where datasets are sparse, and realistic modelling of clean or obfuscated traces are required.

Bibliography

- [1] Baker, K. (2023, February 27). 12 types of malware + examples that you should know. CrowdStrike. <https://www.crowdstrike.com/en-us/cybersecurity-101/malware/types-of-malware/>.
- [2] Bloxberg, D. (2024, August 29). What is malware?: Risks and protection. VIPRE. https://vipre.com/glossary-terms/what-is-malware-risks-and-protection/?srsltid=AfmBOopmXNxAAIb-q0IQjZuAhUStd47H-0eTuXQyTLf0zMb_HhPs8pXz.
- [3] Le Blanc, P., Ulfert, A. S., Peeters, M., Rispens, S., & Scherer, S. (2024). How emerging technologies shape the future of work. *European Journal of Work and Organizational Psychology*, 33(2), 115–119. <https://doi.org/10.1080/1359432X.2024.2324937>.
- [4] Brody, R. G., Chang, H. U., & Schoenberg, E. S. (2018). Malware at its worst: death and destruction. *International Journal of Accounting and Information Management*, 26(4), 527–540. <https://doi.org/10.1108/IJAIM-04-2018-0046>.
- [5] AV-TEST - The Independent IT-Security Institute. (2025). Atlas - Malware & PUA. AV. <https://portal.av-atlas.org/malware>.
- [6] Shabtai, A., Moskovitch, R., Feher, C., Dolev, S., & Elovici, Y. (2012). Detecting unknown malicious code by applying classification techniques on OpCode patterns. *Security Informatics (Berlin)*, 1(1), Article 1. <https://doi.org/10.1186/2190-8532-1-1>.

- [7] Kinable, J., & Kostakis, O. (2011). Malware classification based on call graph clustering. *Journal of Computer Virology and Hacking Techniques*, 7(4), 233–245. <https://doi.org/10.1007/s11416-011-0151-y>.
- [8] Oprișă, C., Cabău, G., & Sebestyen Pal, G. (2016). Malware clustering using suffix trees. *Journal of Computer Virology and Hacking Techniques*, 12(1), 1–10. <https://doi.org/10.1007/s11416-014-0227-6>.
- [9] Apel, M., Bockermann, C., & Meier, M. (2009). Measuring similarity of malware behavior. 2009 IEEE 34th Conference on Local Computer Networks, 891–898. <https://doi.org/10.1109/LCN.2009.5355037>.
- [10] Vásárhelyi, B. M. (2013). Suffix trees and their applications (Doctoral dissertation, PhD dissertation, Faculty of Science, Budapest).
- [11] Hosseinzadeh, S., Rauti, S., Laurén, S., Mäkelä, J.-M., Holvitie, J., Hyrynsalmi, S., & Leppänen, V. (2018). Diversification and obfuscation techniques for software security: A systematic literature review. *Information and Software Technology*, 104, 72–93. <https://doi.org/10.1016/j.infsof.2018.07.007>.
- [12] Wang, H., Cui, B., Yuan, Q., Shi, R., & Huang, M. (2024). A review of deep learning based malware detection techniques. *Neurocomputing (Amsterdam)*, 598, Article 128010. <https://doi.org/10.1016/j.neucom.2024.128010>.
- [13] Avhankar, M. S., Pawar, J., & Kumbhar, V. A Comprehensive Survey on Polymorphic Malware Analysis: Challenges, Techniques, and Future Directions.
- [14] Gabadinho, A., & Ritschard, G. (2016). Analyzing State Sequences with Probabilistic Suffix Trees: The PST R Package. *Journal of Statistical Software*, 72(3), 1–39. <https://doi.org/10.18637/jss.v072.i03>.
- [15] Mazeroff, G., Gregor, J., Thomason, M., & Ford, R. (2008). Probabilistic suffix models for API sequence analysis of Windows XP applications. *Pattern Recognition*, 41(1), 90–101. <https://doi.org/10.1016/j.patcog.2007.04.006>.
- [16] D. Gibert, C. Mateu, and J. Planes, “The rise of machine learning for detection and classification of malware: Research developments, trends and challenges,” *J. Netw. Comput. Appl.*, vol. 159, Art. 102526, 2020.

- [17] Bilar, D. (2007). Opcodes as predictor for malware. *International Journal of Electronic Security and Digital Forensics*, 1(2), 156–168. <https://doi.org/10.1504/IJESDF.2007.016865>.
- [18] Hu, X., Bhatkar, S., Griffin, K., & Shin, K. G. (2019). MutantX-S: Scalable malware clustering based on static features. *Proceedings of the 2013 USENIX Annual Technical Conference, USENIX ATC 2013*, 187–198.
- [19] Raff, E., Zak, R., Cox, R., Sylvester, J., Yacci, P., Ward, R., Tracy, A., McLean, M., & Nicholas, C. (2018). An investigation of byte n-gram features for malware classification. *Journal of Computer Virology and Hacking Techniques*, 14(1), 1–20. <https://doi.org/10.1007/s11416-016-0283-1>.
- [20] Kita, K., & Uda, R. (2021). Malware Subspecies Detection Method by Suffix Arrays and Machine Learning. *2021 55TH ANNUAL CONFERENCE ON INFORMATION SCIENCES AND SYSTEMS (CISS)*, Article 9400219. <https://doi.org/10.1109/CISS50987.2021.9400219>.
- [21] Shafiq, M. Z., Tabish, S. M., Mirza, F., & Farooq, M. (2009). PE-Miner: Mining Structural Information to Detect Malicious Executables in Realtime. In D. Balzarotti, S. Jha, & E. Kirda (Eds.), *Recent Advances in Intrusion Detection* (Vol. 5758, pp. 121–141). Springer Berlin Heidelberg. https://doi.org/10.1007/978-3-642-04342-0_7.
- [22] Schultz, M. G., Eskin, E., Zadok, E., & Stolfo, S. J. (2001). Data mining methods for detection of new malicious executables. *Proceedings - IEEE Computer Society Symposium on Research in Security and Privacy*, 38–49.
- [23] Gibert, D., Totosis, N., Patsakis, C., Le, Q., & Zizzo, G. (2025). Assessing the impact of packing on static machine learning-based malware detection and classification systems. *Computers & Security*, 156, Article 104495. <https://doi.org/10.1016/j.cose.2025.104495>.
- [24] Min-Feng Wang, Yen-Ching Wu, & Meng-Feng Tsai. (2008). Exploiting Frequent Episodes in Weighted Suffix Tree to Improve Intrusion Detection System. *Proceedings, 22nd International Conference on Advanced Information Networking and Applications Workshops/Symposia: Gino-Wan, Okinawa, Japan 25–28 March 2008*, 1246–1252. <https://doi.org/10.1109/WAINA.2008.144>.

- [25] Karim, Md. E., Walenstein, A., Lakhotia, A., & Parida, L. (2005). Malware phylogeny generation using permutations of code. *Journal in Computer Virology*, 1(1–2), 13–23. <https://doi.org/10.1007/s11416-005-0002-9>.
- [26] Miramirkhani, N., Appini, M. P., Nikiforakis, N., & Polychronakis, M. (2017). Spotless Sandboxes: Evading Malware Analysis Systems Using Wear-and-Tear Artifacts. *Proceedings - IEEE Symposium on Security and Privacy*, Article 7958622. <https://doi.org/10.1109/SP.2017.42>.
- [27] Zico Kolter, J., & Maloof, M. A. (2006). Learning to detect and classify malicious executables in the wild. *Journal of Machine Learning Research*, 7, 2721–2744.
- [28] Rieck, K., Holz, T., Willems, C., Düssel, P., & Laskov, P. (2008). Learning and Classification of Malware Behavior. In D. Zamboni (Ed.), *Detection of Intrusions and Malware, and Vulnerability Assessment* (Vol. 5137, pp. 108–125). Springer Berlin / Heidelberg. https://doi.org/10.1007/978-3-540-70542-0_6.
- [29] Perdisci, R., Lee, W., & Feamster, N. (2010). Behavioral clustering of HTTP-based malware and signature generation using malicious network traces. *Proceedings of NSDI 2010: 7th USENIX Symposium on Networked Systems Design and Implementation*, 391–404.
- [30] Shaukat, K., Luo, S., & Varadharajan, V. (2023). A novel deep learning-based approach for malware detection. *Engineering Applications of Artificial Intelligence*, 122, Article 106030. <https://doi.org/10.1016/j.engappai.2023.106030>.
- [31] Tran, T. K., Sato, H., & Kubo, M. (2019). Image-Based Unknown Malware Classification with Few-Shot Learning Models. *2019 Seventh International Symposium on Computing and Networking Workshops (CANDARW 2019)*, Article 8951652. <https://doi.org/10.1109/CANDARW.2019.00075>.
- [32] Vasan, D., Alazab, M., Wassan, S., Safaei, B., & Zheng, Q. (2020). Image-Based malware classification using ensemble of CNN architectures (IMCEC). *Computers & Security*, 92, Article 101748. <https://doi.org/10.1016/j.cose.2020.101748>.
- [33] Mahdavifar, S., Abdul Kadir, A. F., Fatemi, R., Alhadidi, D., & Ghorbani, A. A. (2020). Dynamic Android Malware Category Classification using Semi-

- Supervised Deep Learning. 2020 IEEE Intl Conf on Dependable, Autonomic and Secure Computing, Intl Conf on Pervasive Intelligence and Computing, Intl Conf on Cloud and Big Data Computing, Intl Conf on Cyber Science and Technology Congress (DASC/PICOM/CBDCOM/CYBERSCITECH), Article 9251198. <https://doi.org/10.1109/DASC-PICom-CBDCom-CyberSciTech49142.2020.000094>.
- [34] Wang, P., Tang, Z., & Wang, J. (2021). A novel few-shot malware classification approach for unknown family recognition with multi-prototype modeling. *Computers & Security*, 106, Article 102273. <https://doi.org/10.1016/j.cose.2021.102273>.
 - [35] Deldar, F., & Abadi, M. (2024). Deep Learning for Zero-day Malware Detection and Classification: A Survey. *ACM Computing Surveys*, 56(2), Article 36. <https://doi.org/10.1145/3605775>.
 - [36] Gusfield, D. (1997). *Algorithms on strings, trees, and sequences: computer science and computational biology*. Cambridge University Press.
 - [37] Fox, E. J., Salk, J. J., & Loeb, L. A. (2009). Cancer Genome Sequencing—An Interim Analysis. *Cancer Research* (Chicago, Ill.), 69(12), 4948–4950. <https://doi.org/10.1158/0008-5472.CAN-09-1231>.
 - [38] Kholidy, H. A., Yousof, A. M., Erradi, A., Abdelwahed, S., & Ali, H. A. (2014). A Finite Context Intrusion Prediction Model for Cloud Systems with a Probabilistic Suffix Tree. In D. AlDabass, Colla, A. Pantelous, & M. Vannucci (Eds.), 2014 European Modelling Symposium (No. 7154055). IEEE. <https://doi.org/10.1109/EMS.2014.90>.
 - [39] Gog, S., & Ohlebusch, E. (2013). Compressed suffix trees: Efficient computation and storage of LCP-values. *The ACM Journal of Experimental Algorithmics*, 18, Article 2461327. <https://doi.org/10.1145/2444016.2461327>.
 - [40] Ferhat Ozgur Catak et al. “Deep learning based Sequential model for malware analysis using Windows exe API Calls”. In: *PeerJ Computer Science* 6 (July 2020), e285. ISSN: 2376-5992. DOI: 10.7717/peerj-cs.285. URL: <https://doi.org/10.7717/peerj-cs.285>.

Appendix

Family–Cluster Proportion Heatmaps

All heatmaps are row-normalized and show $P(\text{cluster} \mid \text{family})$ for individual runs. Brighter cells indicate a larger proportion of samples from a given family assigned to the corresponding cluster.

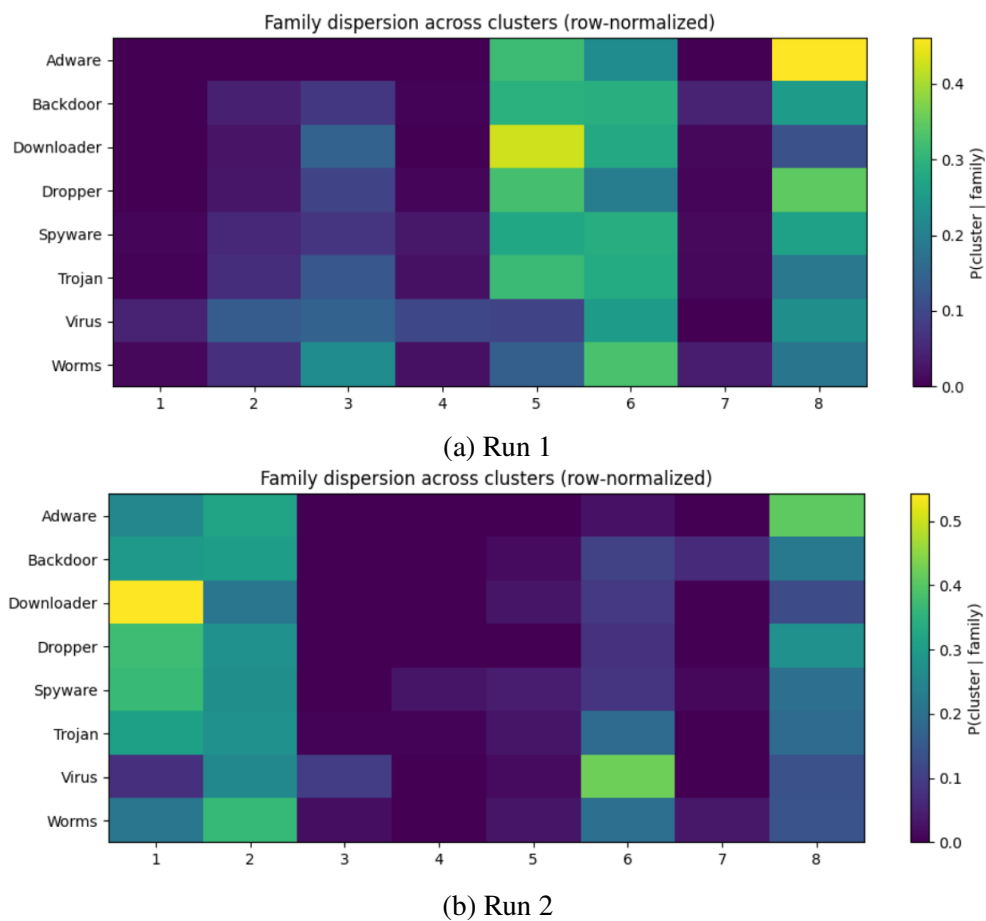
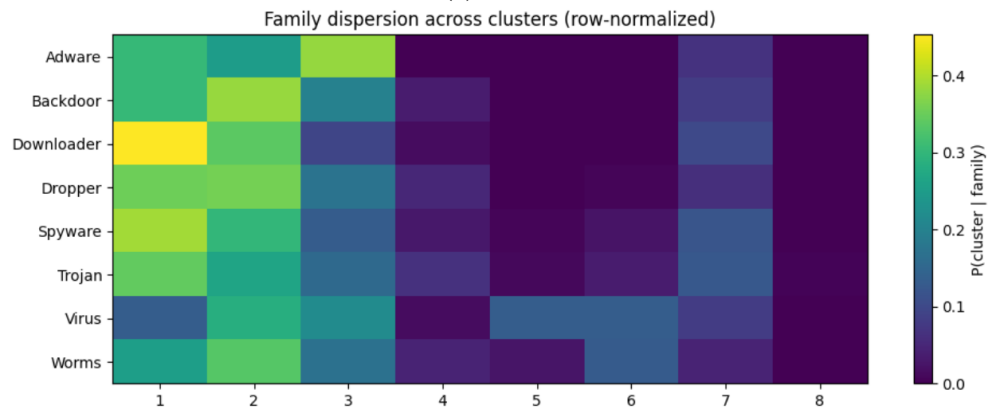


Figure 6.1.1: Family–cluster proportion heatmaps for Runs 1 and 2.



(a) Run 3

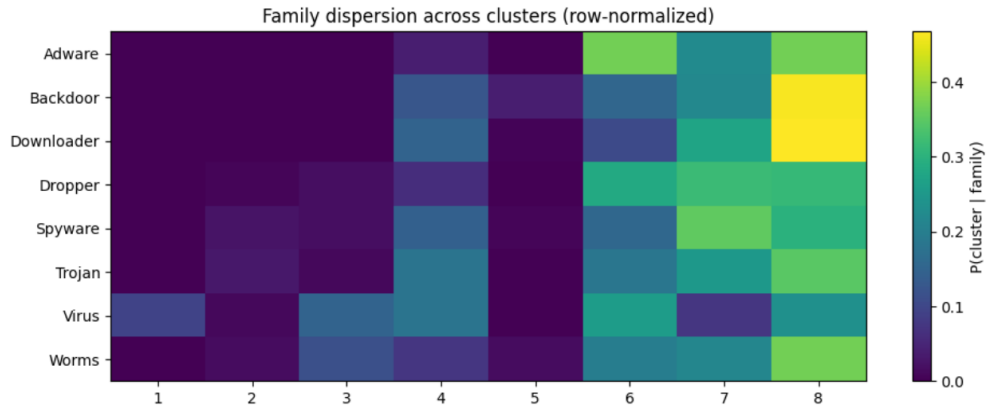


(b) Run 4

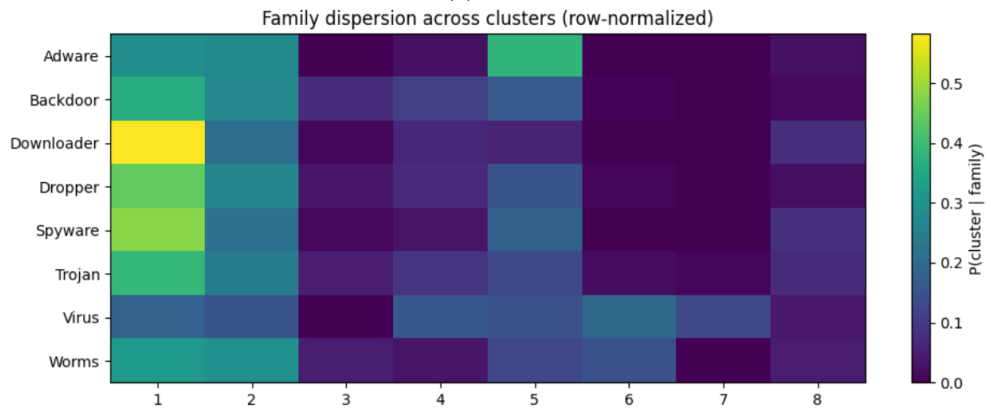


(c) Run 5

Figure 6.1.2: Family–cluster proportion heatmaps for Runs 3, to 5.



(a) Run 6



(b) Run 7

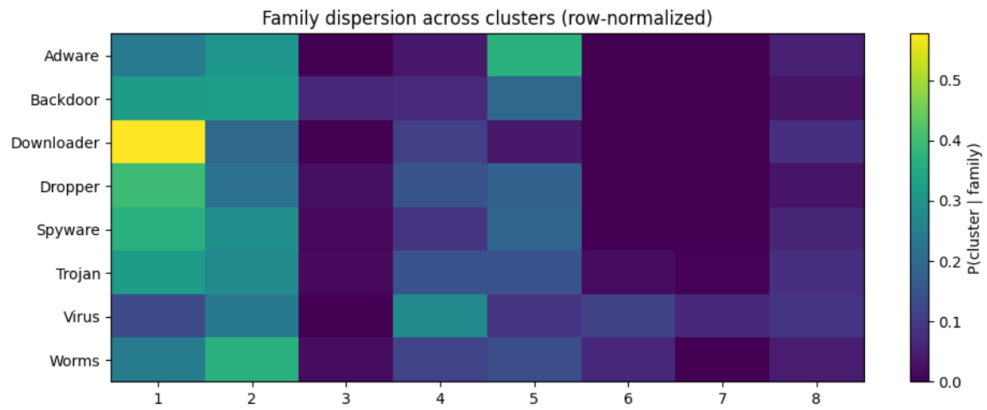


(c) Run 8

Figure 6.1.3: Family–cluster proportion heatmaps for Runs 6, to 8.



(a) Run 9



(b) Run 10

Figure 6.1.4: Family–cluster proportion heatmaps for Runs 9 and 10.

ARI, NMI, Silhouette Scores across runs

Table 6.1.1: ARI, NMI, and Silhouette Scores

Run	ARI	NMI	Global Silhouette
1	0.01590	0.05483	0.20356
2	0.03015	0.07539	0.23760
3	0.01526	0.07366	0.27567
4	0.01271	0.05729	0.26091
5	0.02112	0.08445	0.27472
6	0.01576	0.06779	0.23272
7	0.02126	0.08112	0.24864
8	0.01466	0.06263	0.27695
9	0.01508	0.06541	0.28504
10	0.02414	0.06718	0.24108